Differentiable Gray–Scott Reaction–Diffusion in WebAssembly

Rohit Kumar

***Abstract*—Reaction–diffusion systems are compact nonlinear benchmarks for browser simulation, visualization, and inverse parameter recovery. This paper presents a Rust and WebAssembly implementation of the Gray–Scott model with deterministic validation against Python and NumPy references, render-path measurements, WebAssembly SIMD acceleration, and differentiable inverse recovery for the feed and kill parameters. On the two-parameter inverse task, forward-mode gradients cost about $2.6\times$ a primal loss evaluation on production-relevant native grids, compared with about $5.0\times$ for central finite differences. The WASM SIMD build achieves $7.0\times$–$8.6\times$ speedup over scalar WASM in Node.js. The backtracking AD optimizer reaches lower clean field loss than a dense grid search while using 17 evaluations rather than 1581 candidates, and it completes a 64×64 inverse recovery in 68 ms in-browser via a Web Worker, without GPU, Python, or server-side compute. Noise sweeps show stable grid recovery through uniform noise amplitude 0.020 and degradation by 0.050–0.100. These results support WebAssembly as a practical target for interactive differentiable reaction–diffusion experiments while clarifying the limits of two-parameter forward-mode AD and single-browser performance measurements.**

***Index Terms*—Reaction–diffusion, Gray–Scott model, WebAssembly, automatic differentiation, inverse problems, scientific visualization.**

I. Introduction

REACTION–DIFFUSION systems provide a useful middle ground between toy numerical kernels and full scientific simulation codes. They are small enough to run interactively in a browser, but their nonlinear dynamics make validation, visualization, and inverse recovery nontrivial. The Gray–Scott model [1] is especially attractive because simple changes in the feed parameter F and kill parameter k produce visibly different pattern regimes.

This work asks whether a browser-deliverable implementation can support more than visualization: can it also expose enough numerical structure for credible inverse experiments? The implementation targets Rust for native validation and WebAssembly deployment [2]. The inverse task is to recover F and k from a final simulated pattern under a fixed initial condition.

The contributions are:

- a deterministic Rust/WebAssembly Gray–Scott solver validated against scalar Python and NumPy `float32` references;
- a browser render benchmark separating field-to-RGBA conversion, `ImageData`, `putImageData`, and OffscreenCanvas costs;
- a forward-mode AD implementation for two-parameter inverse recovery;
- an empirical comparison of dense grid search, finite differences, fixed step AD, and Armijo backtracking AD under deterministic target noise;
- a WASM `simd128` forward kernel achieving $7.0\times$–$8.6\times$ speedup over scalar WASM across 128^2–512^2 grids in Node.js;
- a browser inverse page executing the full AD optimizer off the main thread via a Web Worker.

R. Kumar is with Lovely Professional University. E-mail: dv6510@gmail.com.

II. Background

A. Reaction–Diffusion Pattern Formation

Reaction–diffusion pattern formation is usually traced to Turing's chemical basis of morphogenesis [3], which showed how diffusion and local reaction kinetics can destabilize a homogeneous state. Pearson later demonstrated that the Gray–Scott system produces a wide range of complex spatiotemporal patterns despite its compact two-species form [4]. Broader reaction–diffusion theory connects these systems to biological pattern formation, nonlinear chemical dynamics, and parameter-space structure [5]–[8]. This makes Gray–Scott a useful benchmark: it has simple equations, but its final patterns are nonlinear enough to make inverse parameter recovery nontrivial.

B. Gray–Scott Model

The Gray–Scott system describes two reacting and diffusing chemical species, commonly written as

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + F(1-u), \tag{1}$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (F+k)v. \tag{2}$$

Here u and v are concentration fields, D_u and D_v are diffusion coefficients, F is the feed rate, and k is the kill rate. This paper uses an explicit finite-difference update with periodic boundaries and a deterministic square seed.

C. Inverse Recovery

Given a target final field $(u^\star, v^\star)$, the inverse objective is mean squared error over both fields:

$$L(F,k) = \frac{1}{2N} \sum_{i=1}^{N} \left[(u_i(F,k) - u_i^\star)^2 + (v_i(F,k) - v_i^\star)^2 \right]. \tag{3}$$

The experiments compare dense grid search, central finite differences, and forward-mode AD for minimizing this objective.

Fig. 1. Developed v-field pattern generated with $F = 0.060$, $k = 0.062$, a 256×256 grid, and 1000 steps. This visualization is separate from the smaller inverse-recovery benchmark and is included to show a recognizable Gray–Scott pattern regime.

D. WebAssembly for Scientific Kernels

WebAssembly was designed as a compact, safe, low-level compilation target for the Web with predictable validation and execution semantics [2], [9]. Prior work positions it as a portable target for performance-sensitive browser applications rather than a replacement for native HPC. This paper uses that positioning directly: Rust is used for reproducible native tests, while the same solver is exported to WebAssembly for browser deployment and rendering measurements.

E. Differentiable PDE and Physics Frameworks

Differentiable physics systems such as DiffTaichi [10], JAX and JAX MD [11], [12], JAX-Fluids [13], [14], and Φ_{Flow} [15] show that automatic differentiation can be integrated into simulation codes for optimization, learning, and inverse problems. Other work uses differentiable PDE solvers as training components for control and learning [16]–[18]. This paper is narrower: it does not attempt a general differentiable PDE framework, but instead measures a small forward-mode AD path for a browser-deliverable Gray–Scott solver.

F. Browser PDE Tools

Interactive browser PDE tools already show the value of immediate simulation and visual exploration. VisualPDE provides rapid browser-based PDE simulations for education and exploration [19]. The contribution here is complementary: instead of emphasizing general interactive modeling, the system couples a WebAssembly Gray–Scott implementation with native validation, render-path measurements, and a small differentiable inverse-recovery benchmark.

III. Implementation

A. Solver

The solver is implemented in Rust with `f32` fields. The same core update is used for native validation and WebAssembly exports. The browser interface exposes copied fields and zero-copy `Float32Array` views into WASM memory. The primal solver stores the two concentration fields in separate contiguous arrays, u and v, with matching scratch arrays for the next time step. This structure-of-arrays layout keeps the five-point Laplacian stencil simple: each update reads the center, left, right, up, and down entries from one field at a time, writes the corresponding next-field entry, and swaps buffers after a complete step. Periodic boundaries are handled by index wrapping at the stencil accesses, so the storage layout is identical for native and WebAssembly builds.

The WebAssembly interface exposes two kinds of field access. The copied accessors return owned JavaScript arrays and are useful for API safety and testing. The zero-copy accessors return `Float32Array` views over the module memory, avoiding a full field copy across the JavaScript–WebAssembly boundary. Because views into WebAssembly memory can be invalidated if memory grows, rendering code recreates the views after operations that may allocate. The benchmarked browser rendering path uses the zero-copy field view as input, converts the selected field to a reusable RGBA buffer, then measures `ImageData` construction and canvas upload separately.

The differentiated solver uses the same grid traversal and boundary handling, but each scalar value carries a primal component and two tangent components. The current implementation keeps this dual representation as a direct, readable extension of the scalar solver rather than introducing a separate optimized tangent-memory layout. This choice favors validation and paper reproducibility; the measured overhead in Section V shows where memory traffic begins to matter.

A separate `simd128` build provides a SIMD-accelerated forward kernel. Interior rows use 4-wide `f32x4` operations; boundary rows use the scalar path to preserve periodic-boundary correctness. The two builds share the same field layout and are validated against each other before benchmarking. Scalar-vs-SIMD maximum field delta is below 8×10^{-7}; the small checksum difference of 3.2×10^{-9} relative error results from SIMD lane operation reordering.

The browser inverse page executes the AD optimizer inside a module Web Worker via `www/inverse_worker.js`, keeping the main browser thread free during recovery. The worker initializes the WASM module independently and returns a JSON result including optimizer history and a `worker: true` flag confirming off-thread execution.

All code, benchmark scripts, and raw results are available at https://github.com/itisrohit/grayscott-wasm.

B. Forward-Mode AD

For the inverse task, each scalar value carries a primal value and two tangent components, one for F and one for k, following standard forward-mode automatic differentiation [20]–[24]. This directly computes $\partial L/\partial F$ and $\partial L/\partial k$ in one differentiated simulation. For two parameters this is expected to be cheaper than central finite differences, which require five primal evaluations per gradient query.

C. Backtracking Optimizer

The AD optimizer uses steepest descent with Armijo backtracking line search [25], [26]. A candidate step starts from $\alpha_0 = 10^{-3}$, shrinks by 0.5, and is accepted when

$$L(x + \alpha p) \leq L(x) + c\alpha \nabla L(x)^T p, \tag{4}$$

with $c = 10^{-4}$ and $p = -\nabla L(x)$. This adds a small number of primal loss evaluations but avoids the brittle learning-rate behavior observed with a fixed step.

IV. Experimental Setup

All inverse experiments use a 64×64 grid, 100 simulation steps, a deterministic centered square seed, $D_u = 0.16$, $D_v = 0.08$, and $dt = 1$. The default off-grid target is $F = 0.06055$, $k = 0.06245$. Dense grid search uses $F \in [0.045, 0.070]$ with 51 samples and $k \in [0.055, 0.070]$ with 31 samples, for 1581 candidates.

Noise sensitivity is tested by adding deterministic independent uniform noise to both target fields and clamping values to $[0, 1]$. Four seeds are used at noise amplitudes 0.000, 0.020, 0.050, and 0.100 for the AD optimizer comparison.

V. Results

A. Correctness Validation

Full-field comparisons against NumPy `float32` references show small absolute error across tested regimes. Table I reports the four 64×64 regimes tested at 1000 steps. The largest observed maximum error is below 4×10^{-5}, occurring in the more sensitive $F = 0.050$, $k = 0.065$ regime.

Table II reports the complementary multi-grid validation for the default regime. Agreement remains tight through 512×512 and 1000 steps, with maximum error below 6×10^{-7} for both fields in this regime. The mean absolute error decreases at larger grids because the fixed-size seed occupies a smaller fraction of the domain.

B. Scalar Solver Performance

Table III compares native Rust, scalar WASM, SIMD WASM, and scalar JavaScript. Scalar WASM is $1.22\times$–$1.27\times$ faster than scalar JavaScript. The SIMD build achieves $7.0\times$–$8.6\times$ over scalar WASM and $4.7\times$–$6.0\times$ over scalar JavaScript in Node.js. These are Node.js measurements; browser SIMD performance may differ by engine and is left to future work.

C. Rendering Measurements

Zero-copy WASM field views remove full-field copy cost. In Node.js, field-to-RGBA conversion remains under 1 ms per frame at 512×512. Manual Chrome measurements, summarized in Table IV, show that field-to-RGBA conversion dominates `ImageData` construction and `putImageData` for the tested browser path. At 512×512, conversion takes 0.817 ms/frame while direct `putImageData` remains below 0.1 ms/frame. An automated headless Chrome run using three repeated trials confirms field-to-RGBA as the dominant cost at 0.779 ms/frame median for 512×512, consistent with the interactive measurement. Headless direct `putImageData` timing is lower than the interactive Chrome result, so the manual and headless tables are kept separate in the public experiment log.

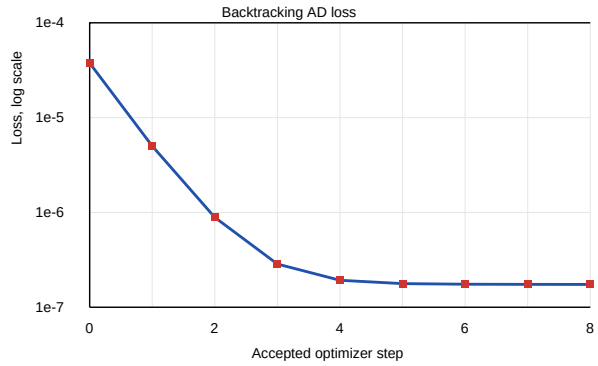

Fig. 2. Backtracking AD optimizer loss trajectory for the clean off-grid target. The nine recorded losses are 3.761×10^{-5}, 5.013×10^{-6}, 8.864×10^{-7}, 2.855×10^{-7}, 1.925×10^{-7}, 1.775×10^{-7}, 1.749×10^{-7}, 1.743×10^{-7}, and 1.740×10^{-7}.

D. Gradient Correctness and Cost

Forward-mode AD is checked directly against central finite differences at multiple off-target inverse points. Table V reports two default-regime guesses, one lower-feed regime, and one farther default-regime guess. The relative deltas remain small for nondegenerate gradients, validating the gradient used by the optimizer.

At 128×128 and 256×256, central finite differences cost about $5.0\times$ a primal loss evaluation, as expected for a two-sided two-parameter gradient plus base loss. Forward-mode AD costs about $2.6\times$ a primal loss evaluation, giving roughly a $2\times$ advantage per gradient query. The overhead is higher than the ideal arithmetic-only estimate, likely because dual fields increase memory traffic and cache pressure.

E. Inverse Recovery

Table VI summarizes the clean off-grid target. The Armijo backtracking AD optimizer reaches lower clean field loss than dense grid search while using far fewer evaluations.

Figure 2 shows the corresponding backtracking loss trajectory. The loss drops quickly during the first few accepted steps and then flattens near 1.74×10^{-7}.

The result also shows why inverse claims must report both field loss and parameter error. Backtracking AD obtains the lowest field loss, but the dense grid candidate is closer to the generating parameters on the clean case.

F. Noise Sensitivity

Grid recovery is stable through noise amplitude 0.020 across four deterministic seeds. At amplitude 0.050, two of four seeds move to a neighboring candidate; at 0.100, three of four seeds move away from the clean candidate. Backtracking AD

TABLE I
FULL-FIELD RUST VS. NUMPY `FLOAT32` VALIDATION AT 1000 STEPS

Regime	u MAE	v MAE	u RMSE	v RMSE	u MaxErr	v MaxErr
$F = 0.037, k = 0.060$	1.697×10^{-7}	1.002×10^{-7}	2.714×10^{-7}	1.794×10^{-7}	1.311×10^{-6}	9.537×10^{-7}
$F = 0.060, k = 0.062$	4.238×10^{-8}	1.755×10^{-8}	8.347×10^{-8}	5.602×10^{-8}	5.960×10^{-7}	5.364×10^{-7}
$F = 0.025, k = 0.060$	1.387×10^{-7}	5.978×10^{-8}	2.560×10^{-7}	1.432×10^{-7}	1.431×10^{-6}	9.984×10^{-7}
$F = 0.050, k = 0.065$	6.430×10^{-7}	3.995×10^{-7}	3.380×10^{-6}	2.707×10^{-6}	3.833×10^{-5}	3.499×10^{-5}

TABLE II
MULTI-GRID FULL-FIELD VALIDATION FOR $F = 0.060, k = 0.062$

Grid	Steps	u MAE	v MAE	u RMSE	v RMSE	u MaxErr	v MaxErr
128^2	100	5.461×10^{-9}	5.200×10^{-10}	2.091×10^{-8}	7.005×10^{-9}	2.384×10^{-7}	1.937×10^{-7}
128^2	500	9.361×10^{-9}	2.535×10^{-9}	3.498×10^{-8}	1.975×10^{-8}	5.364×10^{-7}	3.725×10^{-7}
128^2	1000	1.260×10^{-8}	4.387×10^{-9}	4.402×10^{-8}	2.801×10^{-8}	5.960×10^{-7}	5.364×10^{-7}
256^2	100	1.365×10^{-9}	1.300×10^{-10}	1.045×10^{-8}	3.502×10^{-9}	2.384×10^{-7}	1.937×10^{-7}
256^2	500	2.340×10^{-9}	6.337×10^{-10}	1.749×10^{-8}	9.877×10^{-9}	5.364×10^{-7}	3.725×10^{-7}
256^2	1000	3.150×10^{-9}	1.097×10^{-9}	2.201×10^{-8}	1.400×10^{-8}	5.960×10^{-7}	5.364×10^{-7}
512^2	100	3.413×10^{-10}	3.250×10^{-11}	5.226×10^{-9}	1.751×10^{-9}	2.384×10^{-7}	1.937×10^{-7}
512^2	500	5.850×10^{-10}	1.584×10^{-10}	8.745×10^{-9}	4.938×10^{-9}	5.364×10^{-7}	3.725×10^{-7}
512^2	1000	7.876×10^{-10}	2.742×10^{-10}	1.101×10^{-8}	7.002×10^{-9}	5.960×10^{-7}	5.364×10^{-7}

TABLE III
SCALAR AND SIMD WASM FORWARD SOLVER PERFORMANCE (MS/STEP, 500 STEPS, 5 TRIALS). SIMD BUILD USES `WASM32 SIMD128 F32X4` INTERIOR-ROW VECTORIZATION. NODE.JS MEASUREMENTS.

Grid	Native	Scalar WASM	SIMD WASM	JS	SIMD/JS
128^2	0.050	0.066	0.016	0.084	5.25×
256^2	0.177	0.263	0.055	0.332	6.03×
512^2	0.707	1.053	0.274	1.290	4.71×

TABLE IV
BROWSER RENDER-SIDE MEDIAN TIMING IN CHROME (MS/FRAME)

Grid	RGBA	ImageData	2D putImageData
128^2	0.084000	0.000333	0.010667
256^2	0.205333	0.000000	0.027333
512^2	0.817000	0.000333	0.093333

remains competitive with grid-search noisy loss through 0.100 and keeps lower clean-target loss in the noisy failure cases.

Here "grid clean candidate" means the dense grid returns the same $(F, k) = (0.060500, 0.062500)$ candidate recovered without noise. "AD-line stable" means the backtracking AD optimizer remains in the same final parameter region across all four deterministic seeds.

G. Browser Inverse Performance

The full AD-line inverse loop runs in-browser via a Web Worker without GPU, Python, or server-side compute. Table VIII reports headless Chrome 147 timings for three grid sizes with 8 optimizer iterations and 100 simulation steps. At 64×64, recovery completes in 68 ms after WASM initialization, at 7.6 ms per iteration. The final clean-target loss of 1.740×10^{-7} matches the native result, confirming numerical consistency across the JS/WASM boundary. A `worker: true` flag in the returned status JSON confirms off-thread execution.

VI. DISCUSSION

The experiments support three claims. First, Rust and WebAssembly can host a validated interactive Gray–Scott solver without sacrificing deterministic native testing. Second, forward-mode AD is a practical improvement over central finite differences for a two-parameter inverse query. Third, optimizer mechanics matter: a standard Armijo backtracking rule turns a weak fixed-step AD optimizer into a competitive inverse method.

The current method is not a general differentiable PDE framework. Forward-mode AD scales with the number of parameters, and the $2.6\times$ overhead already shows memory pressure from carrying tangent fields. Larger parameter spaces would motivate adjoint or reverse-mode methods.

VII. LIMITATIONS

The inverse task uses a fixed initial condition, two scalar parameters, and final field MSE. The experiments do not yet establish robustness to unknown initial conditions, observational masks, different time horizons, or real measurement noise. SIMD and browser measurements are from a single machine, with browser measurements running in headless Chrome; interactive and cross-browser validation are left to future work. The browser inverse loop was not tested under network latency or mobile constraints.

VIII. CONCLUSION

This paper presents a validated Rust/WebAssembly Gray–Scott implementation with browser rendering benchmarks, WebAssembly SIMD acceleration, and differentiable inverse recovery. Forward-mode AD gives a measured gradient-cost advantage over finite differences, and Armijo backtracking AD recovers low-loss parameters with far fewer evaluations than a dense grid search. The result is a compact, reproducible platform for interactive differentiable reaction–diffusion experiments. A WASM `simd128` kernel achieves $7\times$–$8.6\times$

TABLE V
AD GRADIENT AGREEMENT WITH CENTRAL FINITE DIFFERENCES

Target	Guess	Loss	F rel.	k rel.	Max rel.
$(0.06055, 0.06245)$	$(0.060, 0.063)$	3.761×10^{-5}	2.418×10^{-4}	3.598×10^{-4}	3.598×10^{-4}
$(0.06055, 0.06245)$	$(0.061, 0.062)$	2.327×10^{-5}	4.646×10^{-4}	1.956×10^{-3}	1.956×10^{-3}
$(0.05025, 0.06025)$	$(0.051, 0.060)$	1.088×10^{-6}	1.054×10^{-3}	2.085×10^{-3}	2.085×10^{-3}
$(0.06055, 0.06245)$	$(0.059, 0.064)$	3.084×10^{-4}	2.142×10^{-4}	9.977×10^{-4}	9.977×10^{-4}

TABLE VI
CLEAN OFF-GRID INVERSE RECOVERY

Method	F	k	Loss	Eval.
Grid search	0.060500	0.062500	2.997×10^{-7}	1581
Fixed-step AD	0.059956	0.062867	1.318×10^{-5}	9
Backtracking AD	0.059894	0.062671	1.740×10^{-7}	17

TABLE VII
NOISE SENSITIVITY SUMMARY ACROSS FOUR SEEDS

Noise	Grid clean candidate	Grid shifted	AD-line stable
0.000	4/4	0/4	4/4
0.020	4/4	0/4	4/4
0.050	2/4	2/4	4/4
0.100	1/4	3/4	4/4

TABLE VIII
BROWSER INVERSE RECOVERY TIMING (HEADLESSCHROME 147, WEB WORKER, 8 ITERATIONS, 100 STEPS)

Grid	Eval.	Elapsed ms	ms/iter	Clean loss
32^2	21	22.2	2.47	6.884×10^{-7}
64^2	17	68.0	7.56	1.740×10^{-7}
96^2	17	142.8	15.87	1.737×10^{-7}

speedup over scalar WASM in Node.js, and the full differentiable inverse loop completes in 68 ms off the main browser thread via a Web Worker for a 64×64 grid.

ACKNOWLEDGMENT

The author thanks the open-source Rust and WebAssembly communities whose tooling made this work possible.

REFERENCES

[1] P. Gray and S. K. Scott, "Autocatalytic reactions in the isothermal, continuous stirred tank reactor: Oscillations and instabilities in the system A + 2B → 3B; B → C," *Chemical Engineering Science*, vol. 39, no. 6, pp. 1087–1097, 1984.

[2] WebAssembly Community Group, "Webassembly core specification, version 2.0," 2022. [Online]. Available: https://www.w3.org/TR/wasm-core-2/

[3] A. M. Turing, "The chemical basis of morphogenesis," *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences*, vol. 237, no. 641, pp. 37–72, 1952.

[4] J. E. Pearson, "Complex patterns in a simple system," *Science*, vol. 261, no. 5118, pp. 189–192, 1993.

[5] J. D. Murray, "Parameter space for turing instability in reaction diffusion mechanisms: A comparison of models," *Journal of Theoretical Biology*, vol. 98, no. 1, pp. 143–163, 1982.

[6] ——, *Mathematical Biology II: Spatial Models and Biomedical Applications*, 3rd ed. New York, NY, USA: Springer, 2003.

[7] A. J. Koch and H. Meinhardt, "Biological pattern formation: From basic mechanisms to complex structures," *Reviews of Modern Physics*, vol. 66, no. 4, pp. 1481–1507, 1994.

[8] I. R. Epstein and J. A. Pojman, *An Introduction to Nonlinear Chemical Dynamics: Oscillations, Waves, Patterns, and Chaos*. New York, NY, USA: Oxford University Press, 1998.

[9] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, D. Gohman, L. Wagner, A. Zakai, J. Bastien, and M. Holman, "Bringing the web up to speed with WebAssembly," *ACM SIGPLAN Notices*, vol. 52, no. 6, pp. 185–200, 2017.

[10] Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley, and F. Durand, "Difftaichi: Differentiable programming for physical simulation," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=B1eB5xSFvr

[11] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: https://github.com/google/jax

[12] S. S. Schoenholz and E. D. Cubuk, "JAX M.D.: A framework for differentiable physics," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2021, no. 12, p. 124016, 2021.

[13] D. A. Bezgin, A. B. Buhendwa, and N. A. Adams, "JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows," *Computer Physics Communications*, vol. 282, p. 108527, 2023.

[14] ——, "JAX-Fluids 2.0: Towards HPC for differentiable CFD of compressible two-phase flows," *Computer Physics Communications*, vol. 308, p. 109433, 2025.

[15] P. Holl and N. Thuerey, "Φ_{flow} (PhiFlow): Differentiable simulations for PyTorch, TensorFlow and JAX," in *International Conference on Machine Learning*. PMLR, 2024. [Online]. Available: https://icml.cc/virtual/2024/37279

[16] P. Holl, V. Koltun, and N. Thuerey, "Learning to control PDEs with differentiable physics," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=HyeSin4FPB

[17] K. Um, R. Brand, Y. Fei, P. Holl, and N. Thuerey, "Solver-in-the-loop: Learning from differentiable physics to interact with iterative PDE-solvers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6111–6122. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/43e4e6a6f341e00671e123714de019a8-Abstract.html

[18] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[19] B. J. Walker, A. K. Townsend, A. K. Chudasama, and A. L. Krause, "VisualPDE: Rapid interactive simulations of partial differential equations," *Bulletin of Mathematical Biology*, vol. 85, no. 113, 2023.

[20] L. B. Rall, *Automatic Differentiation: Techniques and Applications*. Berlin, Germany: Springer, 1981.

[21] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA, USA: SIAM, 2008.

[22] M. Bartholomew-Biggs, S. Brown, B. Christianson, and L. Dixon, "Automatic differentiation of algorithms," *Journal of Computational and Applied Mathematics*, vol. 124, no. 1–2, pp. 171–190, 2000.

[23] B. A. Pearlmutter, "Fast exact multiplication by the hessian," *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994.

[24] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. [Online]. Available: https://jmlr.org/papers/v18/17-468.html

[25] L. Armijo, "Minimization of functions having lipschitz continuous first partial derivatives," *Pacific Journal of Mathematics*, vol. 16, no. 1, pp. 1–3, 1966.

[26] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.